Project Overview

This project is an **Air Quality Regression** machine learning pipeline designed to predict ambient temperature (T) using environmental sensor measurements and air pollutant data from the Air Quality UCI dataset. The workflow spans data ingestion, rigorous preprocessing, exploratory data analysis, traditional gradient boosting, custom polynomial linear regression in PyTorch, and a fully tuned deep neural network optimized using Optuna.

Implemented Components and Features

1. Environment and Reproducibility Setup

- Integrated essential data science and deep learning libraries including NumPy, Pandas, PyTorch, Optuna, Matplotlib, Seaborn, Statsmodels, and Scikit-Learn.
- Configured hardware acceleration dynamically to utilize CUDA, Apple Silicon (MPS), or CPU based on device availability. This accelerates tensor operations and mitigates computational bottlenecks during backpropagation.
- Established a custom `set_seed` function (fixed to seed 42) to enforce deterministic behavior across random modules, NumPy, PyTorch, CUDA, and environment variables. This ensures stochastic optimization trajectories remain strictly invariant across execution runs.

2. Data Handling and Preprocessing

- Loaded the raw data from `AirQualityUCI.csv` using a semicolon delimiter.
- Executed type casting by replacing commas with periods and converting string-encoded columns (`CO(GT)`, `C6H6(GT)`, `T`, `RH`, `AH`) into floating-point numbers.
- Cleaned the dataset by dropping irrelevant columns (`Unnamed: 15`, `Unnamed: 16`, `NMHC(GT)`), dropping entirely null rows, and filtering out records where temperature $T = -200$.
- Addressed missing values (originally denoted as `-200`) by testing an averaging imputation technique before applying linear interpolation across all primary feature columns. Because air quality and meteorological metrics exhibit strong temporal autocorrelation, assigning a global mean severely distorts the localized time-series manifold.
- Linear interpolation, conversely, assumes local linearity between adjacent temporal nodes, preserving the underlying cyclical trajectories of the feature space. This methodological shift effectively attenuates the introduction of synthetic variance and biases that standard averaging would otherwise propagate through the network.

3. Data Splitting

- Defined the feature matrix X by removing `Time`, `Date`, `T`, and `AH`, while setting the target vector y to temperature `T`. Categorical and non-stationary string structures like `Date` and `Time` were excluded to satisfy the prerequisite of purely numerical feature spaces for tensor operations.
- The explicit exclusion of Absolute Humidity (`AH`) was a critical design choice to circumvent deterministic target leakage. Because absolute humidity is functionally dependent on ambient temperature, retaining it would permit the estimator to trivially reverse-engineer the target variable.

- Eradicating such collinear covariates enforces the network to learn latent, non-trivial representations rather than exploiting mathematical tautologies, thereby protecting the model's out-of-sample generalization capabilities.
- Split the dataset into training, validation, and test subsets using nested `train_test_split` functions.

4. Exploratory Data Analysis (EDA)

- Visualized feature relationships by generating a correlation heatmap with Seaborn.
- Plotted feature pairplots and a time-series line chart tracking temperature changes over datetime indices.
- Conducted an Ordinary Least Squares (OLS) regression statistical summary using `statsmodels`.
- Examined feature distributions and outliers using boxplots and histograms.

5. Machine Learning and Deep Learning Models

- **Gradient Boosting Regressor:** Trained a `HistGradientBoostingRegressor` optimized via `GridSearchCV` over hyperparameters such as learning rate, max iterations, depth, and L2 regularization, yielding an optimal MSE of 2.57 and R^2 of 0.96.
- **PyTorch Linear Regression:** Built a custom PyTorch module (`LinearRegressionModel`) using degree-2 `PolynomialFeatures` and `StandardScaler`. Standardizing the feature vectors to a zero-mean and unit-variance distribution mitigates heterogeneous gradient updates, facilitating mathematically stable convergence during gradient descent.
- A strictly linear mapping proved insufficient for encapsulating the intrinsically nonlinear interactions amongst multi-pollutant covariates. Projecting the input space into a second-degree polynomial basis allows the linear hyperplane to effectively model localized curvatures. The model was trained with the Adam optimizer over 30,000 epochs, achieving a test MSE of 4.3390 and an R^2 of 0.9406.
- **Deep Neural Network (MLP):** Developed a Multi-Layer Perceptron (`AirQualityMLP`) featuring two hidden layers and ReLU activation functions.
- **Regularization & Early Stopping:** Implemented a custom `EarlyStopping` class to track validation loss, save optimal checkpoints via `copy.deepcopy`, and introduced data augmentation through Gaussian noise injection (`torch.randn_like * 0.01`) during training batches.
- Injecting isotropic Gaussian noise directly into the input mini-batches acts as an implicit form of Tikhonov regularization. This stochastic perturbation forces the network to converge upon flatter, more robust local minima. Concurrently, early stopping terminates the optimization regime upon detecting plateaued metrics, anchoring weights to the epoch of maximal out-of-sample performance.
- **Optuna Hyperparameter Optimization:** Performed a 20-trial Optuna study to optimize layer sizes, optimizers (`Adam`, `SGD`, `RMSprop`), learning rates, and batch sizes. Utilizing a Bayesian optimization framework circumvents the combinatorial explosion inherent to exhaustive grid search methodologies.
- By leveraging a Tree-structured Parzen Estimator (TPE), the algorithm probabilistically models the objective function and intelligently samples the hyperparameter topology.

This allows for a highly efficient traversal of high-dimensional, non-convex parameter spaces, converging on a competitive architecture with minimal computational overhead.

- **Final Optimized Model:** Deployed the best trial configuration (`units_layer1: 124`, `units_layer2: 66`, `optimizer: RMSprop`, `lr: ~0.0011`, `batch_size: 32`) with an extended patience of 100 epochs, reaching top performance with a final Test MSE of **1.5702** and an R^2 score of **0.9785**.